703159 VU

Data-driven Analysis of Geodata

Supervised classification for permafrost prediction

Dr. Dominik Amschwand, Prof. Jan Beutel

SS2026 @ Dep. of Computer Science
University of Innsbruck

universität innsbruck

CONTENTS

Two approaches to model permafrost distribution:

- Naive Bayes
- Logistic regression

Learning goals

- Understand the test-train-validation data split and why this split is important to assess a model's generalization capability.
- Understand logistic regression under the hood, including the mechanics of gradient descent, and how different parts of the algorithm belong to the representation–evaluation–optimisation components.
- Understand which topo-climatic variables control permafrost distribution, and be able to draw possible causal graphs.
- Understand the assumptions behind `naive Bayes` classifier and how it violates causality.
- Understand binary classifiers, the confusion matrix, and performance metrics.

EXERCISE Causal networks and naive Bayes

- Naive Bayes: MAAT and PISR–Rock glacier activity

Jupyter notebook `.ipynb`

Naive Bayes classifier

Visualisations/graphical models of naive Bayes classifier

Naive representation

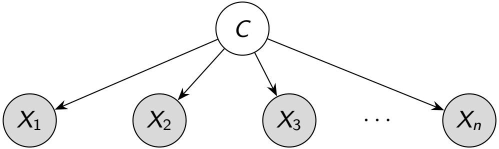
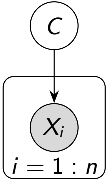


Plate notation



Graphical syntax

- Shaded nodes are observed, open nodes are latent/hidden.
- Variables within a plate are replicated in a conditionally independent manner.

Naive Bayes: Equations (and is it truly Bayesian?)

Class membership $\hat{y}$ is

$$\hat{y} = \underset{k \in \{1,\dots,K\}}{\arg\max}\ p(C_k \mid x_1, x_2, \dots, x_n) \quad \text{(MAP-form decision rule)} \tag{1}$$

$$\propto \underset{k \in \{1,\dots,K\}}{\arg\max}\ p(C_k)\, p(x_1, \dots, x_n \mid C_k) \quad \text{(Bayes thm.)} \tag{2}$$

$$= \underset{k \in \{1,\dots,K\}}{\arg\max}\ p(C_k) \prod_{i=1}^{n} p(x_i \mid C_k) \quad (\text{Factorisation, } X_i \perp\!\!\!\perp X_j \mid C) \tag{3}$$

- The naive Bayes classifier assumes that features are conditionally independent given the class, allowing factorisation of the likelihood (a critical modelling shortcut).
- Naive Bayes gets its name from applying Bayes' theorem, not from doing Bayesian statistics ($\hat{y}$ is not a true MAP estimate, although it formally looks like one; see lecture block on Bayes)!

The model is based on $X_i \perp\!\!\!\perp X_j \mid C$ for tractability. What about $X_i \perp\!\!\!\perp X_j$, or $C \perp\!\!\!\perp X_i$?

Frequentist logistic regression

How training estimates $P(C \mid X)$ in logistic regression.
Given training data $\{x_i, c_i\}_{i=1}^{n}$:

1. **Model assumption (conditional model)**. Sigmoid with parameters $\boldsymbol{\theta}$:

$$P(C = 1 \mid X = x; \theta) = \sigma(\theta^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

2. **Likelihood model:** Each label is treated as a sample from a Bernoulli distribution:

$$c_i \sim \mathrm{Bernoulli}(P(C = 1 \mid x_i; \theta))$$

3. **Conditional likelihood:**

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} P(c_i \mid x_i; \theta) = \prod_{i=1}^{n} \left[\sigma(\theta^\top x_i)\right]^{c_i} \left[1 - \sigma(\theta^\top x_i)\right]^{1-c_i}$$

Frequentist logistic regression

4. **Log-likelihood** (cross-entropy/log-loss objective):

$$\ell(\theta) = \sum_{i=1}^{n} \log P(c_i \mid x_i; \theta) = \sum_{i=1}^{n} \left[c_i \log \sigma(\theta^\top x_i) + (1 - c_i) \log \left(1 - \sigma(\theta^\top x_i)\right) \right]$$

5. **Optimization** is solved numerically (no closed-form solution exists), typically via gradient descent or Newton methods:

$$\hat{\theta} = \arg\max_{\theta} \ \ell(\theta)$$

6. **Prediction** for a new input $\tilde{x}$:

$$\widehat{P}(C = 1 \mid X = \tilde{x}) = \sigma(\hat{\theta}^\top \tilde{x}).$$

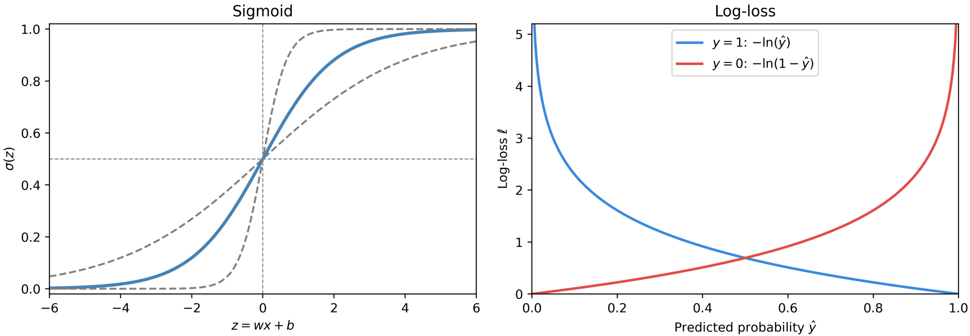
The frequentist estimate is

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_{\theta} P(\boldsymbol{c} \mid \boldsymbol{X}; \theta) \tag{4}$$

- Logistic regression: MAAT–Rock glacier activity

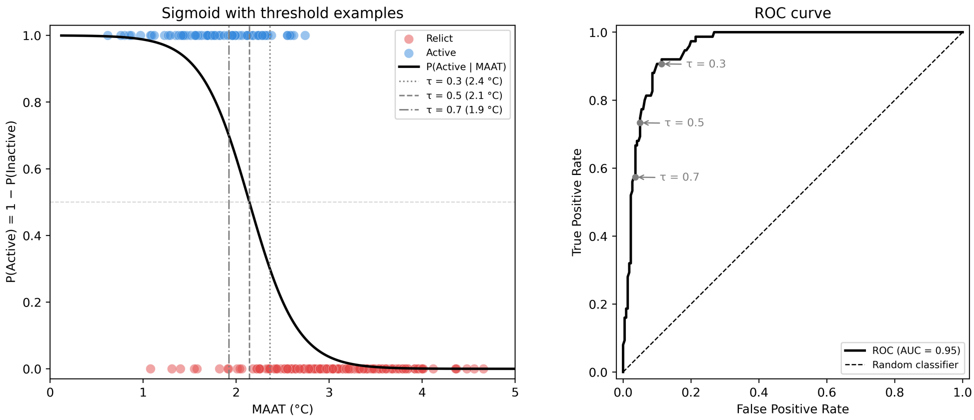
Jupyter notebook `.ipynb`

Binary classification using logistic regression



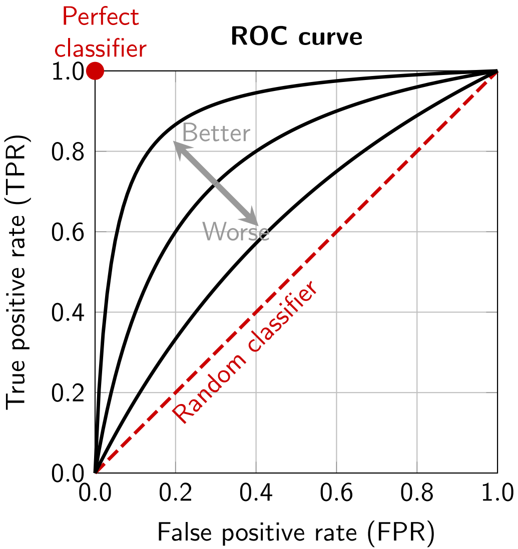
Own work.

Binary classification using logistic regression



Own work, data from Sattler et al. [2016].

ROC curve


ROC curve
Perfect classifier
Better
Worse
Random classifier
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
True positive rate (TPR)
False positive rate (FPR)

‘Performance in production’

Recall the two distinct motivations for ML: prediction and inference.

The goal of predictive modelling is to correctly predict new instances, that is, a high **generalisation ability**. *Model performance must be assessed on unseen data!* Otherwise, the model might overfit the training data.

- Predictive performance evaluated on held-out data
- “Leakage” of test data into modelling process must be avoided – this is more subtle than you might think!
- Data is split into training, validation, and test set (e.g., 60–20–20)

EXAMPLE Autonomous vehicles must cope with a stream of unseen scenes in the real world. “But it worked with the training data” is no excuse after an accident.

QUESTION Consider a linear regression. In which situations is it appropriate to compute R^2 on the ‘training data’, and when not?

EXAMPLE Assessing a linear model

Modelling philosophy does have practical implications. Let's pretend linear regression was a modern ML algorithm (apologies to Sir Francis Galton).

Focus, key question, metric type.

Statistical view	**ML view**
Focus on inference: Explaining relationships in the existing data.	Focus on prediction: Predicting new data.
How well does the model fit/explain the existing data?	How well does the model perform on unseen, new data?
Goodness-of-fit metric calculated on the training data (data used to fit the model, training R^2).	Predictive performance calculated on held-out data (test R^2), generalisation robustness on multiple splits (cross-validated R^2).

Generalisation gap

The **generalisation error** (or **generalisation gap**) is the difference between the training error and the expected error on new, unseen data: $\Delta E = E_{\text{new}} - E_{\text{train}}$.

- The generalisation gap is closely related to the overfitting/underfitting issue.
- The **test error** E_{new} describes how a model performs on new, unseen data.
- The **training error** E_{train} describes how a model performs on the specific data it was trained on.
- Whereas E_{train} measures how well the method *reproduces* its training data, E_{new} tells us how well it performs when put "into production".

Generally, the training error gives **no information** on how well the algorithm will perform for new unseen data points!

QUESTION What is the training error for k-NN with $k = 1$?

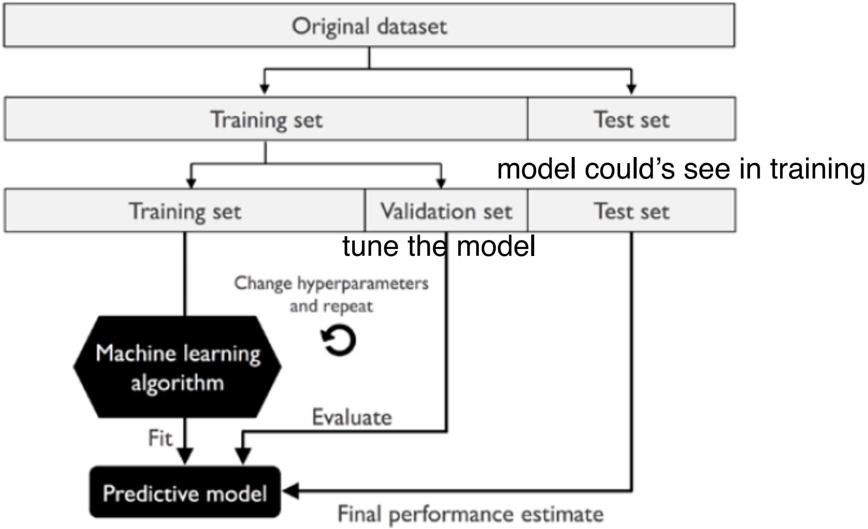
[Lindholm et al., 2022, Chap. 4].

How to measure the test error?

The data set is split into

- A training set, a set of training examples the network is trained on.
- A validation set, which is used to tune hyperparameters.
- A test set, which is used to measure the generalization performance.

ML work flow



[Raschka & Mirjalili, 2020]

Cross-Validation

The problem: The new data is (obviously) not available. How can we estimate E_{new}?

Cross-validation (CV) estimates the generalisation error E_{new} using only the training data, by repeatedly fitting the model on a subset and evaluating on the remainder.

- **Hold-out:** Split the data set once into training, validation, and test data. Train model on the training data, tune hyperparameters on the validation data, and evaluate the model on the test data.
- **k-fold cross-validation:** Partition the training data into k equal folds. In each of k rounds, one fold serves as the validation set and the remaining $k-1$ folds are used for training. The CV estimate is the average error across all k rounds:

$$E_{\text{CV}} = \frac{1}{k}\sum_{i=1}^{k} E_{\text{new}}^{(i)}.$$

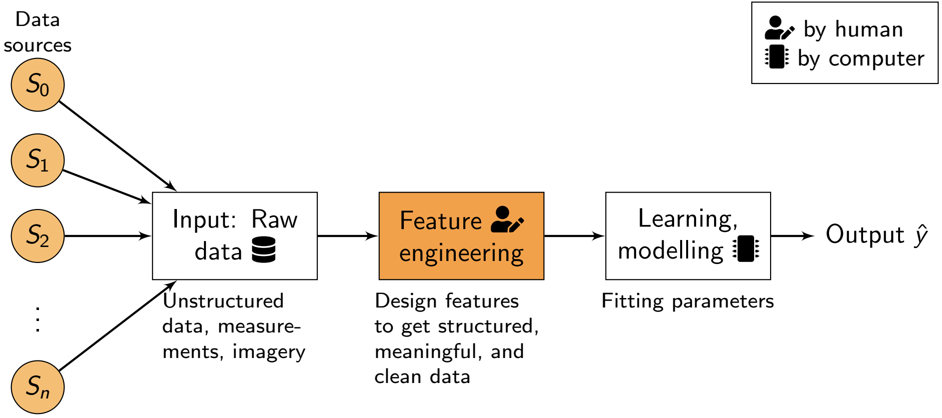
Common choices are $k = 5$ or $k = 10$. Reduces variance compared to hold-out.

Cross-validation

Typical workflow:
$\rightarrow$ Use k-fold CV for model selection and hyperparameter tuning
$\rightarrow$ retrain the final model on the *entire* training set
$\rightarrow$ evaluate once on the held-out **test set**.

uibk course "Data Engineering and Analytics" by E. Zangerle.

Feature Engineering: Making data machine-readable



Why feature engineering?

Feature engineering (FE) is the process of adjusting the representation of the (raw) data to improve the efficacy of the ML algorithms.

- FE is essentially a representation problem.
- FE is the step in the ML workflow where domain knowledge enters
- Good features
  - capture the underlying problem-specific data structure
  - are tailored to the ML algorithm
  - are human-interpretable (if inference rather than prediction is the focus)

[Duboue, 2020].

A taxonomy of feature engineering

Feature engineering is the process of transforming raw data into features that better represent the underlying problem to a machine learning model, improving its accuracy and performance (and sometimes interpretability).

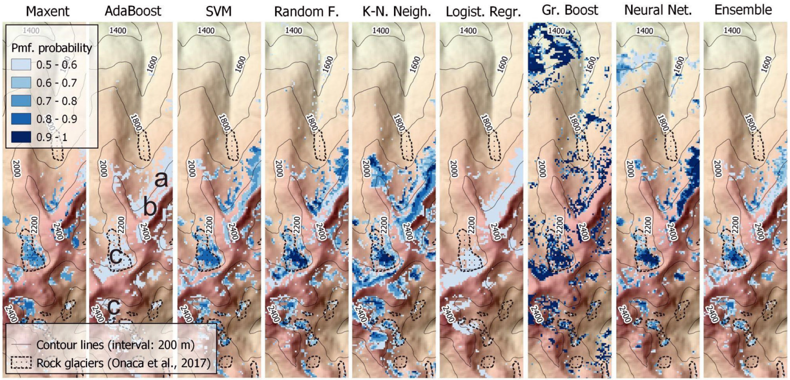
```
Feature engineering
├── Enhancing features
├── Expanding features
└── Reducing features
```

Three types or task groups:

- Enhancing features: Improving existing features without changing the number of them (scaling, log-transformation, encoding, hashing, ...);
- Expanding features: Creating new features from existing ones to expose hidden patterns (imputation, creating interaction terms, ...). **Feature extraction**;
- Reducing features: Shrinking the feature space to remove noise and redundancy (dropping collinear and low-variance features, PCA/SVD, ...). Dimensionality reduction, **feature selection**.

CASE STUDY Predicting permafrost distribution

- Supervised ML/classification problem: From logistic regression to ML
- APPLICATION Permafrost mapping by Popescu et al. [2024].
- Given air temperature and sunshine hours, where is the ground cold enough for permafrost ($T_g \leq 0°\mathrm{C}$ year round)?



[Popescu et al., 2024]

References

Duboue, P. (2020). *The Art of Feature Engineering: Essentials for Machine Learning* (1 ed.). Cambridge University Press.

Lindholm, A., Wahlström, N., Lindsten, F., & Schön, T. B. (2022). *Machine Learning - A First Course for Engineers and Scientists*. Cambridge University Press.

Popescu, R., Filhol, S., Etzelmüller, B., Vasile, M., Pleșoianu, A., Vîrghileanu, M., Onaca, A., Șandric, I., Săvulescu, I., Cruceru, N., Vespremeanu-Stroe, A., Westermann, S., Sîrbu, F., Mihai, B., Nedelea, A., & Gascoin, S. (2024). Permafrost Distribution in the Southern Carpathians, Romania, Derived From Machine Learning Modeling. *Permafrost and Periglacial Processes*, *35*(3), 243–261.

Raschka, S. & Mirjalili, V. (2020). *Python machine learning: machine learning and deep learning with Python, scikit-learn, and TensorFlow 2* (3rd ed ed.). Birmingham: Packt Publishing.

Sattler, K., Anderson, B., Mackintosh, A., Norton, K., & De Róiste, M. (2016). Estimating Permafrost Distribution in the Maritime Southern Alps, New Zealand, Based on Climatic Conditions at Rock Glacier Sites. *Frontiers in Earth Science*, *4*.